# ReUse

Guia passo a passo para colocar o projeto para rodar no seu computador, no Chrome e no celular Android.

Flutter (app) | Dart (API) | MySQL (banco) | VS Code | Windows

| | |
|---|---|
| **Primeira vez** | cerca de 1h30 (a maior parte é esperar downloads e instalações) |
| **Nas próximas vezes** | uns 5 minutos: ligar o banco, ligar a API e rodar o app |
| **Precisa saber programar?** | Não. É só seguir os passos na ordem e copiar os comandos. |

Plataforma para reaproveitamento de materiais

Projeto acadêmico de Engenharia de Software - Universidade de Mogi das Cruzes - 2026

# Como usar este guia

Siga as partes **na ordem**. Cada passo tem um número. Quando aparecer uma caixa preta, é um comando para digitar (ou copiar e colar) no terminal do VS Code e apertar Enter. No final do guia estão os **problemas comuns** (parte 9), **como o código funciona** (parte 10) e uma **folha de comandos rápidos** (parte 11).

TERMINAL - DIGITE

```
PS C:\projetos\ReUse\reuse_api> dart pub get
```

A parte azul (`PS C:\...>`) mostra **em qual pasta** o terminal precisa estar. Você digita só a parte **branca**.

**O QUE DEVE APARECER**

```
Got dependencies!
```

Caixa tracejada verde: o resultado esperado. Se aparecer algo parecido, deu certo.

**ATENÇÃO**

Um cuidado importante para não dar erro.

**DICA**

Algo que facilita a sua vida.

**PRONTO**

Você terminou uma etapa.

**SE DER ERRO**

O que fazer quando algo não sai como esperado.

**COPIAR E COLAR COMANDOS**

Você pode digitar os comandos ou copiar e colar. Para colar no terminal do VS Code use Ctrl + V (ou botão direito). Se copiar do PDF quebrar o comando em duas linhas, copie do arquivo **LEIA-ME.txt** (dentro do zip), que tem todos os comandos.

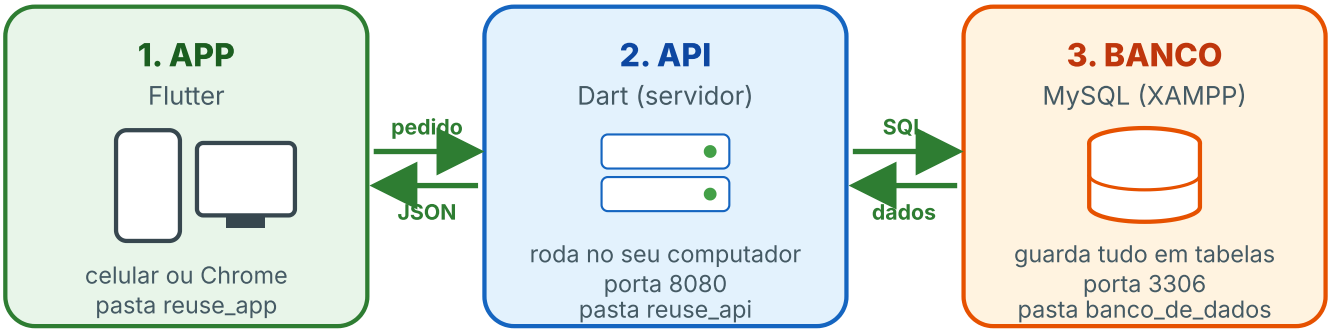
Nas ilustrações, as **bolinhas vermelhas numeradas** 1 mostram onde clicar.

## O caminho completo

1. **Instalar** — VS Code, Git, Flutter, XAMPP e Android Studio. *Só na 1ª vez.*
2. **Descompactar** — Colocar o projeto na pasta certa. *Só na 1ª vez.*
3. **Criar o banco** — Importar o arquivo .sql no phpMyAdmin. *Só na 1ª vez.*
4. **Ligar a API** — O "cérebro" que guarda e entrega os dados.
5. **App no Chrome** — O jeito mais rápido de ver o app funcionando.
6. **App no celular** — Rodar no Android pelo cabo USB.
7. **Gerar o APK** — Instalar no celular sem Play Store.
8. **Apresentar** — Roteiro pronto para a demonstração.

# 0 Entenda o projeto em 1 minuto

São 3 partes que conversam entre si




Para o app funcionar, a **API** e o **banco** precisam estar ligados ao mesmo tempo.

**Exemplo:** quando a artesã toca em *"Solicitar retirada"*, o **app** manda um pedido para a **API**; a API confere os dados e grava no **banco**; o banco confirma e a API responde para o app, que mostra *"Pedido enviado!"*.

## O que vem dentro do arquivo .zip

```
ReUse/
├── banco_de_dados/
│   └── reuse_banco.sql          cria o banco, as tabelas e os dados de exemplo
├── reuse_api/                   a API (back-end em Dart)
│   ├── bin/server.dart          liga o servidor
│   ├── lib/                     rotas, conexão com o banco, configurações
│   └── uploads/                 as fotos dos anúncios ficam aqui
├── reuse_app/                   o aplicativo (front-end em Flutter)
│   ├── lib/                     todas as telas e a IA
│   └── android/                 permissões do Android (internet, localização)
├── Guia_ReUse_Passo_a_Passo.pdf este guia
└── LEIA-ME.txt                  resumo rápido
```

## O que o app faz

| Funcionalidade | Onde fica no app |
|---|---|
| Cadastro e login (pessoa ou empresa) e recuperação de senha | Tela de entrada |
| Anunciar material com foto, tipo, quantidade, estado e local | Botão verde **Anunciar** |
| **Diferencial:** IA analisa a foto e sugere o tipo, ideias de reutilização e possíveis interessados | Botão **Analisar foto com IA** (no anúncio) |

| | |
|---|---|
| Busca com filtros por tipo e distância | Aba **Explorar** |
| Mapa com os anúncios próximos | Aba **Mapa** |
| Solicitar retirada, chat interno e botão de WhatsApp | Detalhe do anúncio e aba **Pedidos** |
| Status: disponível → reservado → retirado | Aba **Pedidos** e detalhe do anúncio |
| Avisos (notificações) | Sininho no canto de cima |
| Painel de impacto (kg de resíduos evitados) | Faixa verde no Explorar e aba **Perfil** |

# 1 Instalar as ferramentas

Só na primeira vez. Se algum programa já estiver instalado, pule o passo.

**ANTES DE COMEÇAR**

Você vai precisar de internet boa e uns **15 GB livres** no disco (o Android Studio é grande). Instale tudo com o usuário administrador do Windows.

### 1 Instale o Visual Studio Code

Acesse **code.visualstudio.com**, clique em **Download for Windows** e instale. Na tela de opções, marque **"Adicionar ao PATH"** e **"Adicionar a ação 'Abrir com Code'"**.

### 2 Instale o Git

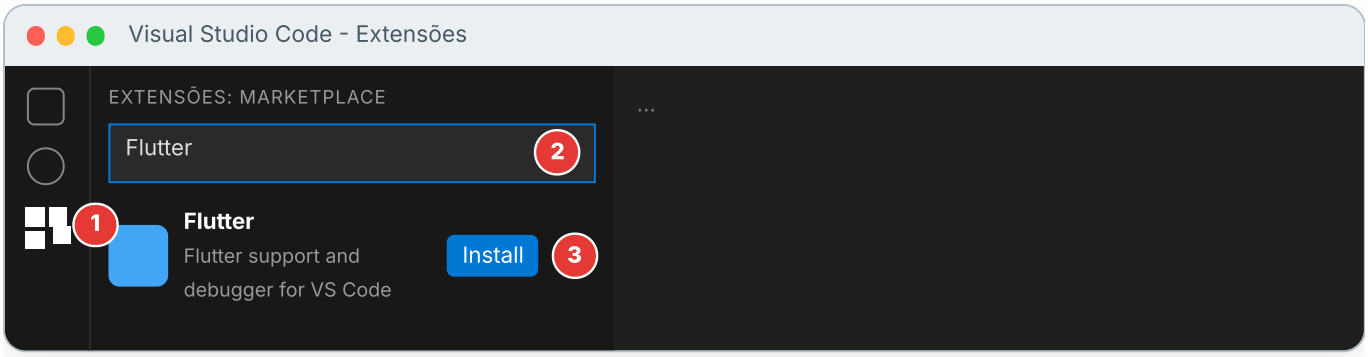
Acesse **git-scm.com/downloads/win**, baixe o instalador e vá clicando em **Next** até o fim (pode deixar tudo como está). O Flutter precisa do Git para ser baixado.

### 3 Instale a extensão Flutter no VS Code

Abra o VS Code. Clique no ícone de **Extensões** 1 (quatro quadradinhos, na barra da esquerda) ou aperte Ctrl + Shift + X. Pesquise **Flutter** 2 e clique em **Install** 3. Ela já instala a extensão **Dart** junto.

**VS CODE EM INGLÊS OU PORTUGUÊS?**

Se o VS Code oferecer o **pacote de idioma Português**, pode aceitar ou não: o guia mostra os nomes dos menus nos dois idiomas.



## 4 Baixe o Flutter pelo próprio VS Code

1. Aperte Ctrl+Shift+P (abre a "paleta de comandos").
2. Digite **flutter** e escolha **Flutter: New Project**.
3. Vai aparecer um aviso dizendo que não achou o Flutter. Clique em **Download SDK**.
4. Escolha a pasta **C:\dev** (crie essa pasta se não existir) e clique em **Clone Flutter**. Espere alguns minutos.
5. Quando terminar, clique em **Add SDK to PATH**. Deve aparecer: *"The Flutter SDK was added to your PATH"*.
6. **Feche e abra o VS Code de novo** (isso é importante).

> **ATENÇÃO COM A PASTA**
>
> Não instale o Flutter em **C:\Program Files** nem em pastas com espaço ou acento (ex.: "Área de Trabalho"). Use **C:\dev**.

## 5 Confira se o Flutter funcionou

No VS Code, abra o terminal pelo menu **Terminal > New Terminal** (em português: **Terminal > Novo Terminal**) (ou Ctrl+') e digite:

TERMINAL - DIGITE

```
PS C:\Users\voce> flutter --version
```

**O QUE DEVE APARECER (os números podem ser maiores)**

```
Flutter 3.47.x • channel stable
Dart 3.13.x • DevTools ...
```

> **SE O FLUTTER JÁ ESTAVA INSTALADO E É ANTIGO**
>
> O projeto precisa do **Flutter 3.35 ou mais novo**. Se o seu for mais velho, atualize com `flutter upgrade`. Se aparecer *"flutter não é reconhecido"*, reinicie o computador e tente de novo.

## 6 Instale o XAMPP (é ele que traz o MySQL)

Acesse **apachefriends.org**, baixe a versão para **Windows** e instale em **C:\xampp**. Na lista de componentes, deixe marcados pelo menos **Apache**, **MySQL** e **phpMyAdmin**. Se aparecer um aviso sobre o "UAC", clique em **OK** e continue.

## 7 Instale o Android Studio (só para rodar no celular e gerar o APK)

Se por enquanto você só quer ver no Chrome, pode deixar este passo para depois.

1. Acesse **developer.android.com/studio**, baixe e instale com as opções padrão.
2. Abra o Android Studio e siga o assistente escolhendo **Standard**. Ele vai baixar o "Android SDK" (demora).
3. Na tela de boas-vindas, clique nos **três pontinhos (More Actions)** > **SDK Manager**.
4. Na aba **SDK Tools**, marque: **Android SDK Command-line Tools**, **Android SDK Build-Tools**, **Android SDK Platform-Tools** e **Google USB Driver**. Clique em **Apply** e depois **OK**.
5. Volte ao terminal do VS Code e aceite as licenças (responda **y** e Enter em todas as perguntas):

TERMINAL - DIGITE

```
PS C:\Users\voce> flutter doctor --android-licenses
```

## 8 Faça o "check-up" final

TERMINAL - DIGITE

```
PS C:\Users\voce> flutter doctor
```

**O QUE DEVE APARECER (mais ou menos assim)**

```
[✓] Flutter (Channel stable, 3.47.x)
[✓] Windows Version
[✓] Android toolchain - develop for Android devices
[✓] Chrome - develop for the web
[✗] Visual Studio - develop Windows apps    <- pode ignorar
[✓] VS Code
[✓] Connected device
```

**O QUE IMPORTA**

Precisam estar com ✓: **Flutter**, **Chrome** e (para celular) **Android toolchain**. O "Visual Studio" serve só para apps de Windows: **pode ignorar** o × dele.

# 2 Descompactar o projeto

Só na primeira vez

### 1 Crie a pasta C:\projetos

Abra o **Explorador de Arquivos**, entre em **Este Computador > Disco Local (C:)** e crie uma pasta chamada **projetos**.

### 2 Extraia o .zip dentro dela

Clique com o botão direito no arquivo **ReUse.zip** > **Extrair tudo...** > escolha **C:\projetos** > **Extrair**.

**COMO DEVE FICAR**

```
C:\projetos\ReUse\banco_de_dados
C:\projetos\ReUse\reuse_api
C:\projetos\ReUse\reuse_app
```

**EVITE ESTAS PASTAS**

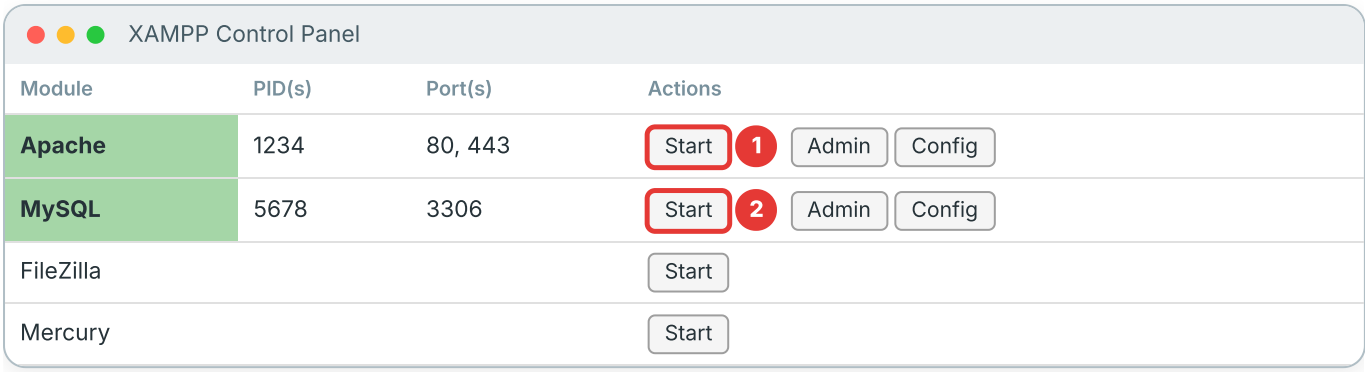
Não use a Área de Trabalho, Downloads ou OneDrive. Caminhos com espaço, acento ou sincronização na nuvem costumam dar erro na hora de gerar o app Android. **C:\projetos** é o lugar mais seguro.

# 3 Criar o banco de dados

Só na primeira vez (ou quando quiser "zerar" os dados)

### 1 Ligue o MySQL no XAMPP

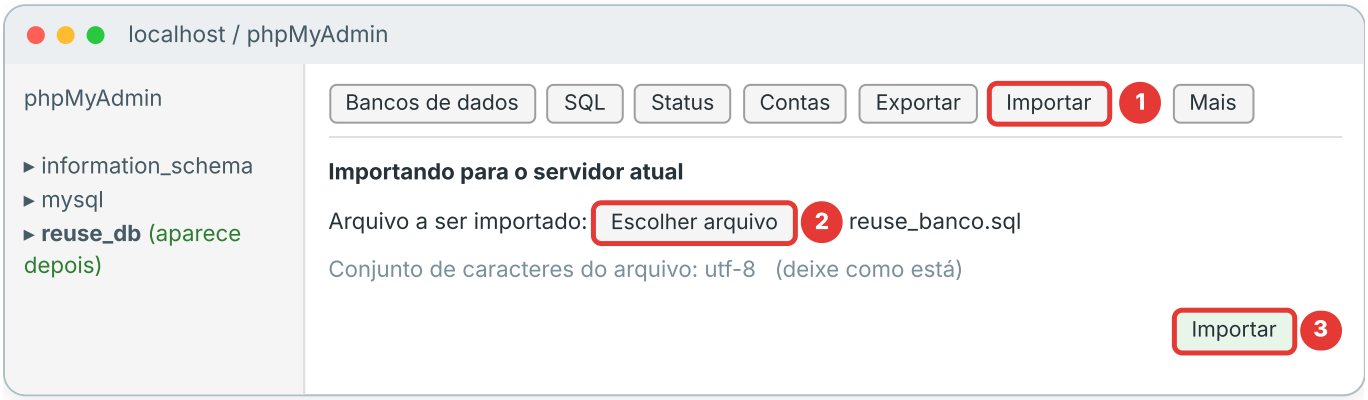
Abra o **XAMPP Control Panel** (procure "XAMPP" no menu Iniciar). Clique em **Start** na linha do **Apache** 1 e na linha do **MySQL** 2. Os nomes ficam com **fundo verde** quando estão ligados.




Quando estão ligados, os módulos ficam verdes e o botão muda para "Stop".

## 2 Abra o phpMyAdmin e importe o arquivo

1. No Chrome, acesse **http://localhost/phpmyadmin**
2. Clique na aba **Importar** 1 (no alto da página).
3. Clique em **Escolher arquivo** 2 e selecione **C:\projetos\ReUse\banco_de_dados\reuse_banco.sql**
4. Role a página até o fim e clique em **Importar** 3.




**O QUE DEVE APARECER**

```
Importação finalizada com sucesso, XX consultas executadas. (reuse_banco.sql)
```

No menu da esquerda vai aparecer o banco **reuse_db** com 8 tabelas: *anuncios*, *empresas*, *mensagens*, *notificacoes*, *recuperacoes_senha*, *sessoes*, *solicitacoes*, *usuarios*.

**CONTAS DE TESTE JÁ CRIADAS (SENHA DE TODAS: 123456)**

**ana@reuse.com** (artesã) • **marcenaria@reuse.com** • **escola@reuse.com** • **confeccao@reuse.com** • **carlos@reuse.com** (serralheiro)

O script também cria o usuário de banco **reuse_app** (senha **ReUse@2026**), que a API usa para entrar no MySQL.

**PRONTO**

O banco está criado. Para "zerar" tudo e voltar aos dados de exemplo, é só importar o arquivo de novo.

# 4 Ligar a API

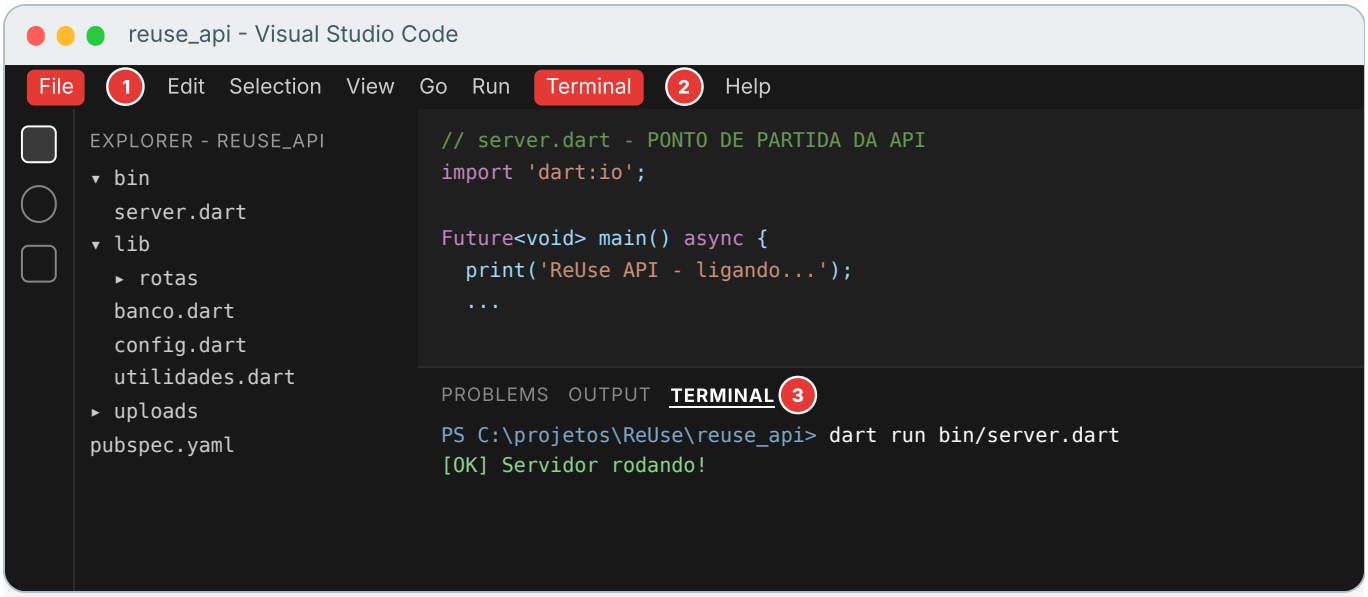
Faça isto sempre que for usar o app (com o MySQL do XAMPP ligado)

### 1 Abra a pasta da API no VS Code

No VS Code: menu **File > Open Folder...** (em português: **Arquivo > Abrir Pasta...**) 1 > escolha **C:\projetos\ReUse\reuse_api** > **Selecionar pasta**. Se perguntar *"Do you trust the authors?"*, clique em **Yes, I trust the authors** (em português: **Sim, confio nos autores**).

### 2 Abra o terminal

Menu **Terminal > New Terminal** (em português: **Terminal > Novo Terminal**) 2. Ele abre embaixo 3, já dentro da pasta certa.




### 3 Baixe os pacotes da API (só na primeira vez)

TERMINAL - DIGITE

```
PS C:\projetos\ReUse\reuse_api> dart pub get
```

**O QUE DEVE APARECER**

```
Resolving dependencies...
...
Got dependencies!
```

## 4 Ligue a API

TERMINAL - DIGITE

```
PS C:\projetos\ReUse\reuse_api> dart run bin/server.dart
```

**O QUE DEVE APARECER**

```
==============================================
  ReUse API - ligando...
==============================================
[OK] Conectado ao banco "reuse_db" em 127.0.0.1:3306
[OK] Servidor rodando!

  No computador (Chrome):  http://localhost:8080
  No celular (mesmo Wi-Fi): http://192.168.0.15:8080   <- anote este!

  Deixe esta janela aberta enquanto usa o app.
```

**AVISO DO FIREWALL DO WINDOWS**

Na primeira vez, o Windows pergunta se o **dart.exe** pode acessar a rede. Marque **Redes privadas** e clique em **Permitir acesso**. Sem isso o celular não consegue falar com a API.

## 5 Teste no navegador

Abra o Chrome e acesse **http://localhost:8080**

**O QUE DEVE APARECER**

```
{"app":"ReUse API","status":"funcionando","hora":"2026-..."}
```

**PRONTO**

A API está no ar. **Não feche esse terminal** enquanto estiver usando o app. Cada pedido que o app fizer vai aparecer listado nele (ótimo para mostrar na apresentação). Para desligar: clique no terminal e aperte Ctrl + C.

# 5 Rodar o app no Chrome

O jeito mais rápido de ver tudo funcionando

### 1 Abra uma SEGUNDA janela do VS Code

A primeira janela continua com a API ligada. Menu **File > New Window** (em português: **Arquivo > Nova Janela**). Na janela nova: **File > Open Folder...** (em português: **Arquivo > Abrir Pasta...**) > **C:\projetos\ReUse\reuse_app**. Depois abra o terminal: **Terminal > New Terminal** (em português: **Terminal > Novo Terminal**).

> **POR QUE DUAS JANELAS?**
>
> Uma janela = API rodando. A outra = app rodando. Assim um não atrapalha o outro.

### 2 Prepare o projeto (só na primeira vez)

Este comando cria as pastas do Android e da Web que o Flutter precisa. Ele **não apaga** nada do código que já está lá.

TERMINAL - DIGITE

```
PS C:\projetos\ReUse\reuse_app> flutter create . --platforms=android,web --org br.umc.reuse
```

**O QUE DEVE APARECER**

```
Recreating project ....
  ...
Resolving dependencies...
Got dependencies.
All done!
```

> **REPARE NO PONTO**
>
> Tem um **ponto ( . )** depois de `create`, separado por espaços. Ele quer dizer "nesta pasta". Não esqueça dele.

## 3 Rode o app no Chrome

TERMINAL - DIGITE

```
PS C:\projetos\ReUse\reuse_app> flutter run -d chrome
```

**O QUE DEVE APARECER**

```
Launching lib\main.dart on Chrome in debug mode...
Waiting for connection from debug service on Chrome...   (pode demorar 1 a 2 minutos)
...
Flutter run key commands.
r Hot reload.
R Hot restart.
q Quit (terminate the application on the device).
```

Uma janela do Chrome abre sozinha com a tela de login do ReUse.

## 4 Entre com uma conta de teste

Toque no botão **Ana (artesã)** (1) embaixo de "Contas de teste" (ele preenche e-mail e senha) e depois em **Entrar** (2). Se o Chrome pedir para usar sua localização, clique em **Permitir**.




Tela de login e aba Explorar (desenho simplificado).

**MUDOU O CÓDIGO? VEJA NA HORA**

Salve o arquivo (Ctrl + S) e aperte **r** no terminal do app (hot reload). Para reiniciar o app: **R**. Para fechar: **q**.

**OUTRA FORMA: BOTÃO DE "PLAY"**

Em vez do comando, você pode escolher **Chrome** no canto de baixo à direita do VS Code (onde aparece o dispositivo) e apertar F5.

**PRONTO**

O ReUse está rodando! Nas próximas vezes, basta: XAMPP (MySQL) ligado > `dart run bin/server.dart` na API > `flutter run -d chrome` no app.

# 6 Rodar o app no celular Android

Pelo cabo USB. Precisa do Android Studio (parte 1, passo 7)

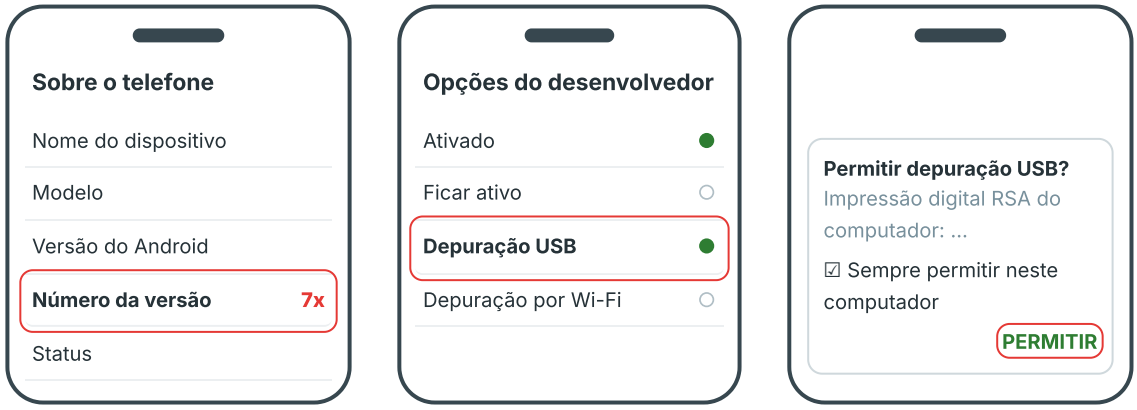
### 1 Ative o "modo desenvolvedor" do celular

No celular: **Configurações > Sobre o telefone** e toque **7 vezes** em **Número da versão** (ou "Número de compilação"). Vai aparecer: *"Você agora é um desenvolvedor"*.

Samsung: Configurações > Sobre o telefone > Informações do software > Número de compilação. Xiaomi: Configurações > Sobre o telefone > Versão do MIUI / HyperOS.

### 2 Ligue a Depuração USB

Em **Configurações > Sistema > Opções do desenvolvedor** (o caminho muda um pouco em cada marca), ative **Depuração USB**.




1) toque 7 vezes  2) ligue a Depuração USB  3) ao conectar o cabo, permita.

### 3 Conecte o cabo e confira

Ligue o celular no computador com um cabo USB **que transmita dados** (alguns cabos só carregam). Aceite a pergunta **"Permitir depuração USB?"** no celular, marcando **Sempre permitir**.

TERMINAL - DIGITE (na janela do app)

```
PS C:\projetos\ReUse\reuse_app> flutter devices
```

**O QUE DEVE APARECER (com o nome do seu celular)**

```
Found 3 connected devices:
  moto g54 (mobile) • ZY22ABCDEF • android-arm64 • Android 15 (API 35)
  Chrome (web)      • chrome     • web-javascript • Google Chrome
  ...
```

## 4 Rode o app no celular

TERMINAL - DIGITE

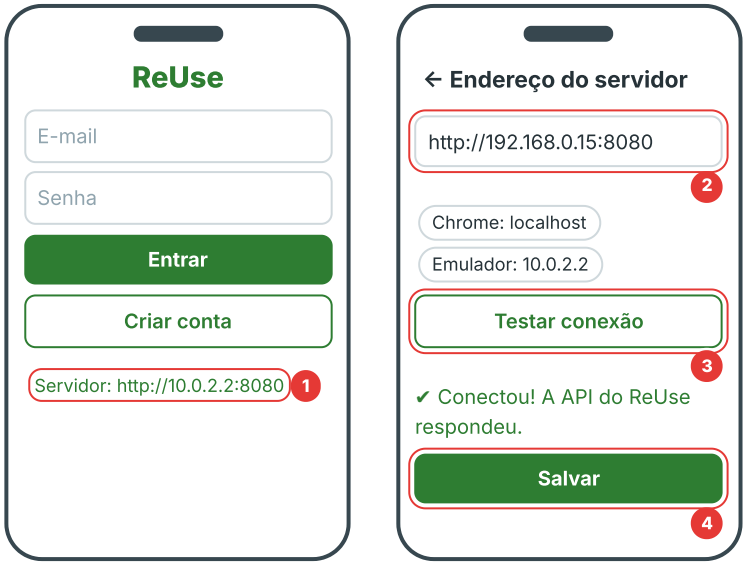
```
PS C:\projetos\ReUse\reuse_app> flutter run
```

Se ele perguntar qual dispositivo usar, digite o **número** do seu celular e Enter. **A primeira vez demora** (5 a 10 minutos, ele baixa o "Gradle"). As próximas são bem mais rápidas.

## 5 Aponte o app para a API do seu computador

No celular, o endereço *localhost* não funciona (ele é o próprio celular). Por isso:

1. Confira que **o celular e o computador estão no mesmo Wi-Fi**.
2. No terminal da **API**, copie o endereço da linha **"No celular (mesmo Wi-Fi)"**, ex.: `http://192.168.0.15:8080`
3. No app, na tela de login, toque em **Servidor: ...** (embaixo de tudo) 1.
4. Digite o endereço 2, toque em **Testar conexão** 3. Deve aparecer *"Conectou!"*. Toque em **Salvar** 4.




**SE O TESTE DISSER "NÃO CONECTOU"**

- A API está ligada? (o terminal dela precisa mostrar "Servidor rodando").
- Você permitiu o **dart.exe** no Firewall? Se não apareceu o aviso, procure no Windows por **"Permitir um aplicativo pelo Firewall do Windows"** e marque o **dart.exe** em **Privada**.
- O Wi-Fi da faculdade às vezes bloqueia a comunicação entre aparelhos. Teste com o **roteador de casa** ou crie um **hotspot no celular** e conecte o computador nele (aí use o novo IP que aparecer no terminal da API, reiniciando a API).

# 7 Gerar o APK e instalar sem Play Store

O arquivo .apk é o "instalador" do app para Android

### 1 Gere o APK

TERMINAL - DIGITE

```
PS C:\projetos\ReUse\reuse_app> flutter build apk --release
```

**O QUE DEVE APARECER (no final)**

```
√ Built build\app\outputs\flutter-apk\app-release.apk (25.0MB)
```

### 2 Encontre o arquivo

Ele fica em **C:\projetos\ReUse\reuse_app\build\app\outputs\flutter-apk\app-release.apk**. Atalho: no VS Code, abra essa pasta no Explorer da esquerda, clique com o botão direito em **app-release.apk** > **Reveal in File Explorer** (em português: **Revelar no Explorador de Arquivos**).

### 3 Mande para o celular e instale

1. Envie o arquivo por **WhatsApp** (pode mandar para você mesmo), **Google Drive**, e-mail ou cabo USB.
2. No celular, toque no arquivo. O Android vai pedir para **permitir instalar apps desta fonte**: toque em **Configurações**, ative a opção e volte.
3. Toque em **Instalar**. Se o **Play Protect** avisar, toque em **Mais detalhes > Instalar mesmo assim** (é normal para apps que não estão na loja).

**LEMBRE-SE**

O APK é só o **app**. Ele continua precisando da **API e do banco ligados no seu computador**, com o celular no **mesmo Wi-Fi**. Na primeira vez que abrir o app instalado, configure o **Servidor** (parte 6, passo 5). O mesmo .apk pode ser instalado no celular dos colegas.

# 8 Roteiro para a apresentação

Mostra todas as funcionalidades em uns 8 minutos

**DOIS USUÁRIOS AO MESMO TEMPO**

Abra o app no **celular** com uma conta e no **Chrome** com outra. Ou, só no computador: copie o endereço do app no Chrome (ex.: `http://localhost:5xxxx`) e cole numa **janela anônima** (Ctrl + Shift + N): cada janela fica com um login diferente.

| # | Quem | O que fazer e falar |
|---|---|---|
| 1 | Marcenaria<br>marcenaria@reuse.com | Toque em **Anunciar**, tire/escolha a foto de uma sobra de madeira e toque em **Analisar foto com IA**. Mostre o tipo identificado, a confiança, as **ideias de reutilização** e os **possíveis interessados**. Toque em **Usar esta sugestão**, marque o local e publique. |
| 2 | Ana (artesã)<br>ana@reuse.com | Na aba **Explorar**, filtre por **Madeira** e por **distância**. Abra a aba **Mapa**: cada cor é um tipo de material. |
| 3 | Ana | Abra o anúncio novo e toque em **Solicitar retirada**, informando o destino (ex.: "porta-retratos artesanais"). Mostre o botão de **WhatsApp**. |
| 4 | Marcenaria | O **sininho** mostra "Nova solicitação". Na aba **Pedidos > Recebidos**, toque em **Aceitar**: o anúncio fica **Reservado**. Abra **Conversar** e combine a retirada pelo chat. |
| 5 | Marcenaria | Toque em **Confirmar retirada**: o anúncio vira **Retirado**. |
| 6 | Qualquer um | Abra o **Painel de impacto** (faixa verde do Explorar ou Perfil): o total de kg evitados aumentou. Mostre a relação com o **ODS 12**. |
| 7 | Extra | Mostre o terminal da API registrando cada pedido e as tabelas no phpMyAdmin sendo preenchidas (ex.: tabela *solicitacoes*). |

**SOBRE A IA (SEJA TRANSPARENTE NA APRESENTAÇÃO)**

A IA deste protótipo é **simulada e funciona sem internet**: ela analisa as **cores da foto** (ex.: tons de marrom indicam madeira, cinza indica metal) e consulta uma **base de conhecimento** com ideias de reutilização. Por isso pode errar, e o usuário confirma ou escolhe outra opção. Como evolução futura, a mesma função pode ser trocada por um modelo de visão computacional de verdade.

# 9 Problemas comuns e soluções

Procure a mensagem ou o sintoma na coluna da esquerda

| O que aconteceu | Como resolver |
|---|---|
| **"flutter" (ou "dart") não é reconhecido como comando** | O Flutter não está no PATH. Feche e abra o VS Code. Se continuar, reinicie o computador. Se ainda assim não der, refaça a parte 1, passo 4, clicando em **Add SDK to PATH**. |
| **"Because reuse_app requires SDK version ^3.9.0, version solving failed"** | Seu Flutter é antigo. Rode `flutter upgrade` e tente de novo. |
| **O MySQL do XAMPP não fica verde** / "Port 3306 in use" | Já existe outro MySQL ligado no computador. Aperte Win + R, digite `services.msc`, procure "MySQL" e clique em **Parar**. Depois clique em Start no XAMPP de novo. |
| **[ERRO] Não consegui conectar no banco de dados** (no terminal da API) | 1) O MySQL está verde no XAMPP? 2) Você importou o **reuse_banco.sql**? 3) Se aparecer *"Access denied for user 'reuse_app'"*, importe o .sql de novo (ele recria esse usuário). |
| **"Failed to create server socket"** / porta 8080 em uso | A API já está ligada em outro terminal (ou outra janela do VS Code). Feche o outro terminal (lixeira no painel do terminal) ou aperte Ctrl + C nele. |
| **No app: "Não consegui falar com o servidor"** | A API está ligada? No Chrome o servidor deve ser `http://localhost:8080`. No celular, use o IP mostrado no terminal da API (parte 6, passo 5) e confira o Firewall e o Wi-Fi. |
| **O celular não aparece** no `flutter devices` | Troque o cabo (tem que ser de dados) e a porta USB. Desbloqueie a tela e aceite "Permitir depuração USB". Confira se o **Google USB Driver** foi instalado (parte 1, passo 7). Na notificação do USB do celular, escolha **Transferência de arquivos**. |
| **Android toolchain com ×** / "cmdline-tools component is missing" | No Android Studio: SDK Manager > SDK Tools > marque **Android SDK Command-line Tools** > Apply. Depois rode `flutter doctor --android-licenses`. |
| **A primeira execução no Android demora muito** | É normal (5 a 10 minutos): ele baixa ferramentas do Android. Precisa de internet. Não feche o terminal. |
| **Erro do Gradle** ao rodar no Android ou gerar o APK | Confira se o projeto está em **C:\projetos** (sem espaços/acentos e fora do OneDrive). Rode `flutter clean` e depois tente de novo. Para ver detalhes: `flutter doctor -v`. |
| **O mapa aparece cinza / em branco** | O mapa (OpenStreetMap) precisa de **internet**. Confira a conexão do aparelho. |
| **A localização não funciona** | No Chrome: clique no cadeado ao lado do endereço > Localização > Permitir. No celular: ligue a localização e permita para o ReUse. Se não der, use **Escolher no mapa** no anúncio. |
| **As fotos não aparecem** | As fotos vêm da API: confira se ela está ligada e se o endereço do servidor está certo. Elas ficam salvas em **reuse_api\uploads**. |
| **Esqueci a senha** de uma conta que criei | Use "Esqueci minha senha" no app. É um modo de demonstração: o código aparece na tela e no terminal da API. |
| **Linhas vermelhas no código** do VS Code | Normalmente é porque os pacotes não foram baixados. Rode `flutter pub get` (no app) ou `dart pub get` (na API). |
| **Quero recomeçar do zero** | Dados: importe o **reuse_banco.sql** de novo. App: rode `flutter clean` e depois `flutter pub get`. |

# 10 Como o código funciona

Para estudar e explicar o projeto. Todo arquivo tem comentários explicando cada parte.

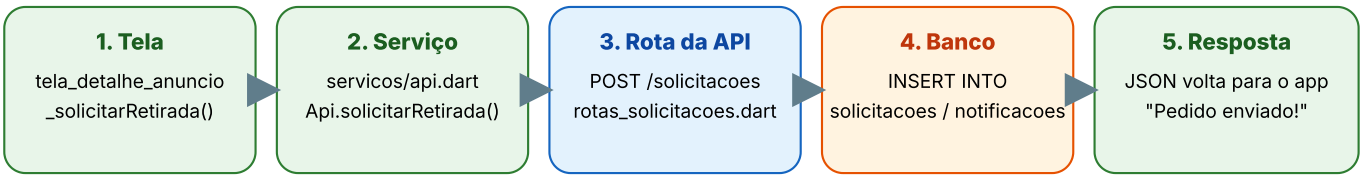
## Mapa das pastas do app (reuse_app/lib)

| Pasta / arquivo | Para que serve |
|---|---|
| `main.dart` | Ponto de partida. Liga o app e abre a primeira tela. |
| `tema.dart` | Cores, tipos de material (ícone e cor) e textos dos status. **Quer mudar a cor do app? É aqui.** |
| `config.dart` | Endereço padrão da API e as contas de teste. |
| `telas/` | Uma tela por arquivo: login, cadastro, feed (Explorar), mapa, detalhe, formulário do anúncio, pedidos, chat, perfil, impacto, avisos... |
| `servicos/api.dart` | **Toda** a conversa com a API. As telas só chamam, por exemplo, `Api.listarAnuncios()`. |
| `servicos/sessao.dart` | Guarda o login e o endereço do servidor no aparelho. |
| `servicos/localizacao.dart` | Pega a localização (GPS ou do navegador). |
| `modelos/` | A "forma" dos dados: Usuario, Anuncio, Solicitacao, Mensagem... |
| `ia/assistente_ia.dart` | O **diferencial**: análise da foto e a base de conhecimento de reutilização. |
| `widgets/` | Peças reaproveitadas em várias telas (card do anúncio, selo de status, painel da IA). |

## Mapa das pastas da API (reuse_api)

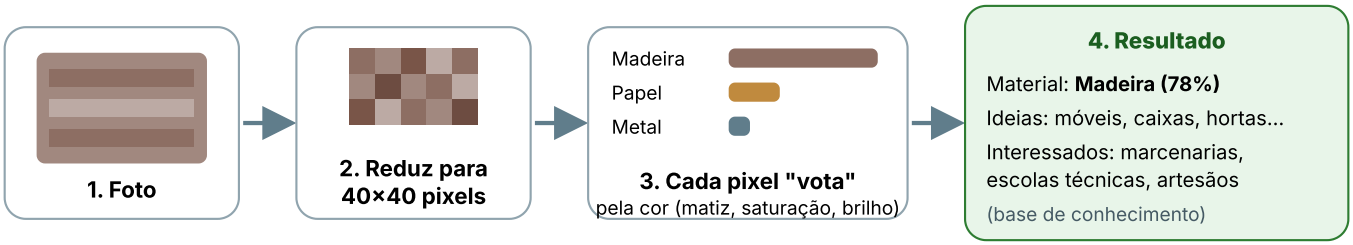
| Arquivo | Para que serve |
|---|---|
| `bin/server.dart` | Liga o servidor, junta as rotas, libera o acesso do Chrome (CORS) e trata os erros. |
| `lib/config.dart` | Porta da API e dados de acesso ao banco. **Se o MySQL for diferente, mude aqui.** |
| `lib/banco.dart` | Conexão com o MySQL. Funções `consultar` (SELECT) e `executar` (INSERT/UPDATE/DELETE). |
| `lib/utilidades.dart` | Respostas em JSON, conferência de login (token), senha com hash SHA-256 e cálculo de distância. |
| `lib/rotas/` | Um arquivo por assunto: usuários, anúncios, solicitações (com chat), notificações e impacto. |

## O caminho de um clique: "Solicitar retirada"




Todas as funcionalidades seguem esse mesmo caminho: Tela → Api → Rota → Banco → Resposta.

## Como funciona a IA (diferencial)




Tudo acontece no próprio aparelho, sem internet. O usuário confirma ou escolhe outro tipo.

## Rotas da API (endereços)

| Método | Endereço | O que faz |
|---|---|---|
| GET | `/` | Rota de teste ("funcionando") |
| POST | `/usuarios/cadastro` `/usuarios/login` | Criar conta / entrar (devolve o token) |
| POST | `/usuarios/recuperar-senha` `/redefinir-senha` | Recuperação de senha com código de 6 dígitos |
| GET / PUT | `/usuarios/me` | Ver / editar o perfil |
| GET | `/anuncios?tipo=&busca=&lat=&lng=&raio_km=` | Feed e mapa, com filtros |
| GET | `/anuncios/meus` `/anuncios/<id>` | Meus anúncios / detalhe |
| POST / PUT / DELETE | `/anuncios` `/anuncios/<id>` | Criar (com foto) / editar / excluir |
| PUT | `/anuncios/<id>/status` | disponível / reservado / retirado |
| POST / GET | `/solicitacoes` | Pedir retirada / listar recebidos e enviados |
| PUT | `/solicitacoes/<id>/aceitar\|recusar\|concluir\|cancelar` | Andamento do pedido |
| GET / POST | `/solicitacoes/<id>/mensagens` | Chat interno |

| | | |
|---|---|---|
| GET / PUT | `/notificacoes` `/notificacoes/lidas` | Avisos do sininho |
| GET | `/impacto` | Painel de impacto (kg evitados) |
| GET | `/uploads/<arquivo>` | Fotos dos anúncios |

## Tabelas do banco (reuse_db)

| Tabela | O que guarda |
|---|---|
| **usuarios** | Todas as contas (pessoa ou empresa). A senha fica guardada como *hash*, nunca pura. |
| **empresas** | Dados extras de empresas (CNPJ, ramo). Ligada a *usuarios*. |
| **anuncios** | Materiais: tipo, quantidade, peso, estado, foto, local (latitude/longitude) e status. |
| **solicitacoes** | Pedidos de retirada e o *destino do material* (finalidade). |
| **mensagens** | Chat de cada pedido. |
| **notificacoes** | Avisos do sininho. |
| **sessoes** / **recuperacoes_senha** | Tokens de login e códigos de recuperação de senha. |

## Quer mudar algo? Onde mexer

| Quero... | Arquivo |
|---|---|
| Mudar a cor principal do app | `reuse_app/lib/tema.dart` (Cores.verde) |
| Adicionar um tipo de material | `tema.dart` (app) + `rotas_anuncios.dart` (API) + `assistente_ia.dart` (ideias da IA) |
| Mudar textos da IA (ideias e interessados) | `reuse_app/lib/ia/assistente_ia.dart` (parte "BASE DE CONHECIMENTO") |
| Mudar a porta da API ou a senha do banco | `reuse_api/lib/config.dart` |
| Mudar as contas de teste da tela de login | `reuse_app/lib/config.dart` |

# 11 Folha de comandos rápidos

Imprima esta página e deixe do lado do computador

## Todo dia (nesta ordem)

1

**XAMPP**

Clique em **Start** no MySQL (e no Apache, se for abrir o phpMyAdmin).

2

**Janela da API**

VS Code com a pasta **reuse_api**: `dart run bin/server.dart`

3

**Janela do app**

VS Code com a pasta **reuse_app**: `flutter run -d chrome` ou `flutter run`

## Todos os comandos usados

| Comando | Onde | Para que serve |
|---|---|---|
| `flutter --version` | qualquer pasta | Ver a versão do Flutter |
| `flutter doctor` | qualquer pasta | Check-up da instalação |
| `flutter doctor --android-licenses` | qualquer pasta | Aceitar as licenças do Android |
| `flutter upgrade` | qualquer pasta | Atualizar o Flutter |
| `dart pub get` | reuse_api | Baixar os pacotes da API |
| `dart run bin/server.dart` | reuse_api | Ligar a API |
| `flutter create . --platforms=android,web --org br.umc.reuse` | reuse_app | Preparar o app (só na 1ª vez) |
| `flutter pub get` | reuse_app | Baixar os pacotes do app |
| `flutter devices` | reuse_app | Ver celulares e navegadores conectados |
| `flutter run -d chrome` | reuse_app | Rodar no Chrome |
| `flutter run` | reuse_app | Rodar no celular |
| `flutter build apk --release` | reuse_app | Gerar o instalador .apk |
| `flutter clean` | reuse_app | Limpar arquivos gerados (quando der erro estranho) |

## Teclas úteis

| Tecla | O que faz | Tecla | O que faz |
|---|---|---|---|
| r no terminal do app | Aplica a mudança do código (hot reload) | Ctrl + C no terminal da API | Desliga a API |
| R no terminal do app | Reinicia o app | Ctrl + ' | Abre/fecha o terminal |
| q no terminal do app | Fecha o app | Ctrl + Shift + P | Paleta de comandos |

**Contas de teste** (senha **123456**): ana@reuse.com • marcenaria@reuse.com • escola@reuse.com • confeccao@reuse.com • carlos@reuse.com